Petri Nets (P/T nets)

NECKER

November 21, 2025

Contents

1 Introduction

Petri nets are a formal language used to model concurrent, distributed, and asynchronous systems. Unlike FSMs (finite state machines):

- The state is **distributed** (global marking = token distribution across places)
- Multiple "local states" can be active simultaneously
- Compact representation of systems characterized by concurrency and shared resources

Mainly used in **critical** (safety-critical) contexts for formal specification and verification.

2 Formal Definition (Classical P/T Net)

A Petri net is a 5-tuple $\mathcal{N} = (P, T, F, W, M_0)$ where:

Symbol	Meaning
P	Finite set of **places** (circles)
T	Finite set of **transitions** (rectangles)
$F \subseteq (P \times T) \cup (T \times P)$	Flow relations (arcs)
$W : F \to \mathbb{N}^+$	Arc weights (1 if not specified)
$M_0 : P \to \mathbb{N}$	**Initial marking** (number of tokens per place)

Conditions: $P \cap T = \emptyset, \;\; P \cup T \neq \emptyset$

2.1 Useful Notation

The presets of a transition represent the nodes from which arrows originate towards that transition; the postsets represent the nodes where the arrows of the transition land.

- ${}^\bullet x = \text{Pre}(x) = \{y \mid (y, x) \in F\}$ (x's preset)
- $x^\bullet = \text{Post}(x) = \{y \mid (x, y) \in F\}$ (x's postset)
- $M(p)$ = number of tokens in place p within marking M

3 Net Dynamics

- A transition t is **enabled** in M (denoted as $M[t\rangle$) if:

$$\forall p \in {}^\bullet t \quad M(p) \geq W(\langle p, t\rangle)$$

- The **firing** of t yields M', denoted as $M[t\rangle M'$:

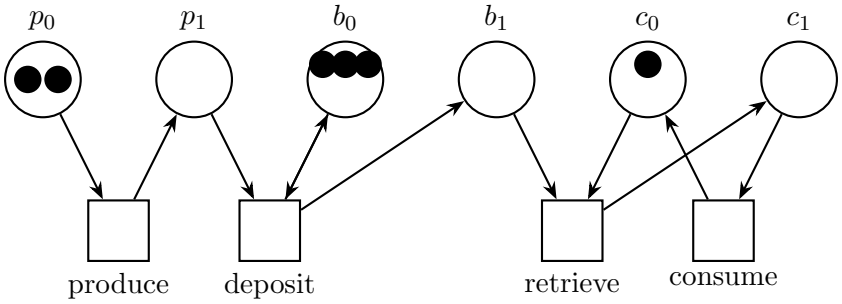
$$M'(p) = \begin{cases} M(p) - W(\langle p, t\rangle) + W(\langle t, p\rangle) & \text{if } p \in {}^\bullet t \cap t^\bullet \\ M(p) - W(\langle p, t\rangle) & \text{if } p \in {}^\bullet t \setminus t^\bullet \\ M(p) + W(\langle t, p\rangle) & \text{if } p \in t^\bullet \setminus {}^\bullet t \\ M(p) & \text{otherwise} \end{cases}$$

In words:

1. if the node is both in the preset and postset
2. if the node is only in the preset
3. if the node is only in the postset
4. if the node is neither in the postset nor in the preset

Analysis Property: to evaluate whether t is enabled, it is sufficient to inspect only the places in ${}^\bullet t$.

4 Graphical Example (Producer-Consumer-Buffer)



5 Relationships Between Transitions: Sequence, Conflict, Concurrency

Petri nets allow classifying the type of interaction between two transitions t_1 and t_2. Two levels of analysis are always distinguished:

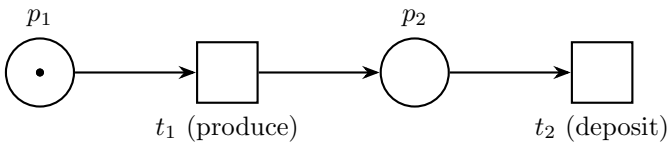
- **Structural** depends exclusively on the net topology (arcs)
- —Actual— depends on the current marking (tokens present)

5.1 1. Sequence (Causality)

The firing of t_1 enables t_2 (at least along certain execution paths).

—Actual sequence— within a marking M:

$$M[t_1\rangle \ \wedge \ \neg M[t_2\rangle \ \wedge \ \exists M' \ M[t_1\rangle M'[t_2\rangle$$



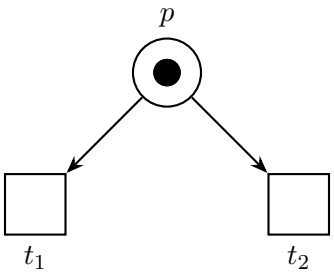
5.2 2. Conflict

Structural Conflict: occurs if two transitions share at least one node within their presets:

$$^\bullet t_1 \cap {}^\bullet t_2 \neq \emptyset$$

—Actual conflict— within M: occurs if two transitions in structural conflict are enabled, but the sum of tokens they require is strictly lower than the current marking at that node:

$$M[t_1\rangle \wedge M[t_2\rangle \wedge \exists p \in {}^\bullet t_1 \cap {}^\bullet t_2 : M(p) < W(p,t_1) + W(p,t_2)$$



5.3 3. Concurrency (Parallelism)

Structural Concurrency: the opposite of conflict; occurs if two transitions share no nodes within their presets:

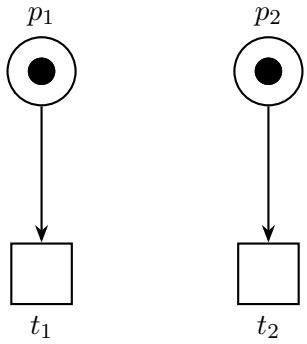
$$^\bullet t_1 \cap {}^\bullet t_2 = \emptyset$$

—Actual concurrency— within M: if two transitions are enabled, and if a node belongs to the preset of both, it must contain enough tokens to activate both simultaneously:

1.
$$M[t_1\rangle \wedge M[t_2\rangle$$

2.
$$\forall p \in Pre(t_1) \cap Pre(t_2)\ M(p) \geq W(\langle p, t_1\rangle) + W(\langle p, t_2\rangle)$$

(if they are structurally concurrent, the second condition is automatically satisfied; moreover, we can have various combinations, e.g., structural and actual concurrency, non-structural but actual concurrency, etc.)



5.4 Summary of the Three Relations

Relation	Structural Level	Actual Level (in marking M)
Sequence	—	$M[t_1\rangle \wedge \neg M[t_2\rangle \wedge \exists M' : M[t_1\rangle M'[t_2\rangle$
Conflict	$^\bullet t_1 \cap {}^\bullet t_2 \neq \emptyset$	$M[t_1\rangle \wedge M[t_2\rangle$ **and** insufficient tokens for both
Concurrency	$^\bullet t_1 \cap {}^\bullet t_2 = \emptyset$	$M[t_1\rangle \wedge M[t_2\rangle$

Rule of Thumb:

- **Share** at least one input place possible **conflict**
- **Do not share** input places possible **concurrency**
- Firing one enables the other **sequence**

6 Core Properties

6.1 Reachability Set and Boundedness

- **Reachability Set** $R(\mathcal{N}, M_0)$ The set containing **all** markings reachable starting from the initial marking M_0 via any (finite) firing sequence of transitions.

  Inductive definition (the smallest set satisfying):

$$M_0 \in R(\mathcal{N}, M_0)$$
$$\text{if } M' \in R(\mathcal{N}, M_0) \text{ and } \exists\, t \in T \text{ such that } M'[t\rangle M'' \implies M'' \in R(\mathcal{N}, M_0)$$

In this case, if there exists a marking M' activated from M_0 and it activates M'', then M'' belongs to the set.
In simple terms:
«start from M_0 and fire any transitions you want, in whatever order you want, as many times as you want: all the resulting token distributions reside in $R(\mathcal{N}, M_0)$».

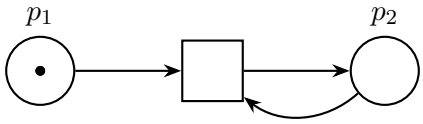
- **Boundedness** A net $(\mathcal{N}, M_0)$ is **bounded** if there exists a finite upper bound of tokens that no place can ever exceed:

$$\exists\, k \in \mathbb{N} \quad \forall M' \in R(\mathcal{N}, M_0) \quad \forall p \in P \quad M'(p) \leq k$$

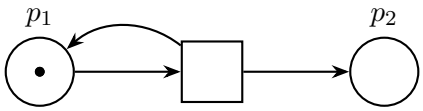
Practical Meaning

  - The token count in *each* place is always bounded from above.
  - $R(\mathcal{N}, M_0)$ has finite cardinality $\iff$ the net is bounded.
  - If bounded, we can build the reachability graph (equivalent automaton).
  - If *not* bounded $\Rightarrow$ there exists at least one place where we can pump infinite tokens.

Immediate Examples



Bounded (max 1 token)



Unbounded (each firing adds a token to p_2)

- **Fundamental Consequence**

$$\text{bounded net} \iff |R(\mathcal{N}, M_0)| < \infty \iff \text{there exists an equivalent finite state automaton}$$

For this reason, boundedness is the primary property verified when attempting to automatically analyze a Petri net.

6.1.1 Liveness of a Transition, Conservativeness, and Reversibility

- **Liveness of a transition** t:
  - Degree 0 (dead): never enabled.
  - Degree 1: enabled at least once.
  - Degree 2: given any positive n, there is a firing sequence enabling it to fire n times (can be fired as many times as desired without bounds, thus infinitely, but it can be blocked by another transition, meaning it might cease to be infinite).
  - Degree 3: fires infinitely many times in at least one execution path (degree 2 does not guarantee infinity in every path).
  - Degree 4 (absolutely live): from any reachable marking, there exists a firing sequence enabling it to fire at least once.

- **Conservativeness** (weighted conservation property) Let $H : P \to \mathbb{N}^+$ be a function mapping each place to a positive weight (e.g., $H(p) = 5$ if p represents 5 units of a resource).

  The net $(\mathcal{N}, M_0)$ is **conservative** with respect to H if the total weighted token value is invariant:

$$\forall M' \in R(\mathcal{N}, M_0) \qquad \sum_{p \in P} H(p) \cdot M'(p) \;=\; \sum_{p \in P} H(p) \cdot M_0(p) \;:=\; k$$

Intuitive Meaning: every transition destroys and creates tokens such that the "total weighted cost" of the system never changes (the weighted sum of each marking is identical).

Fundamental Consequence:

$$\text{conservative} \implies \text{bounded}$$

- **Strict Conservativeness** A special case where all places have a weight of 1:

$$H(p) = 1 \quad \forall p \in P$$

Then the condition simplifies to:

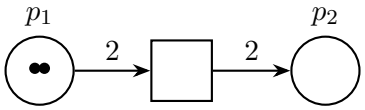
$$\forall M' \in R(\mathcal{N}, M_0) \qquad \sum_{p \in P} M'(p) \; = \; \sum_{p \in P} M_0(p)$$

meaning the **total number of tokens in the net remains constant** forever.

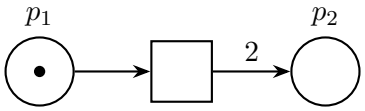
This equates to saying that **every transition consumes exactly as many tokens as it produces**:

$$\forall t \in T \qquad \sum_{p \in {}^\bullet t} W(\langle p, t \rangle) \; = \; \sum_{p \in t^\bullet} W(\langle t, p \rangle)$$

Graphical Examples



Strictly Conservative (always exactly 2 tokens)



Conservative (with $H(p_1) = 2$ and $H(p_2) = 1$) but **not** strictly conservative

- **Implication Relationships**

$$\text{strictly conservative} \implies \text{conservative} \implies \text{bounded}$$

The inverse implications **do not** hold.

- **Reversibility:**
A net is reversible if it is possible to return to the initial base marking from any reachable marking.

7 Extensions and Reductions of Classical Petri Nets

- **Finite Place Capacity** Each place p has a maximum capacity $C(p)$. A transition t can fire only if, after firing, no output place exceeds its capacity limitation. **Equivalence**: yes, it can be translated into a classical P/T net by adding a complementary place initialized with $C(p)$ initial tokens (works only for pure nets, i.e., those without bidirectional arcs between the same place and the same transition).

- **Inhibitor Arcs** (denoted by a small circle on the arc) The transition is enabled only if the source place contains fewer than n tokens (usually n=0). **Equivalence with classical P/T net**:
  - if the place is bounded → yes (via a complementary place)
  - if the place is unbounded → no, the net becomes strictly more powerful (Turing-complete)

- **Arc Weights** ($W > 1$) every net featuring transitions with weights strictly greater than 1 can be rewritten more verbosely as an equivalent net with weights = 1.

- **Condition/Event Nets (C/E)** A highly restrictive sub-class:
  - all arcs have a weight of 1
  - each place has a maximum capacity of 1 (boolean condition)

  Equivalence: any **bounded** P/T net can be translated into an equivalent C/E net. C/E nets cannot model infinite state spaces.

- **Other Widely Used Extensions** (names only)
  - Colored Petri Nets
  - Reset arcs
  - Priority transitions
  - Timed / stochastic Petri nets

8 Petri Net Analysis

Objective: answer typical concurrent/distributed systems questions **before writing actual code**:

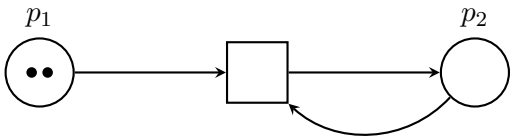
- Is a certain marking reachable?
- Is a specific firing sequence feasible?
- Is deadlock possible?
- Is the net (or a specific transition) live? At what level?
- Is the net bounded? Reversible? Covered by invariants?

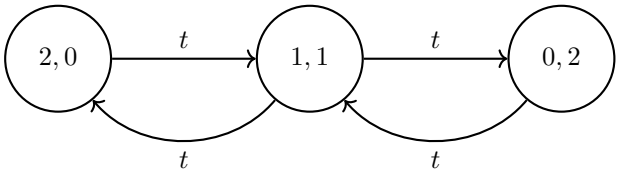
8.1 Available Techniques

Dynamic Techniques (explore states)	**Static Techniques** (topology only)
Reachability Graph/Tree	P-invariants (place invariants)
Coverability Graph/Tree	T-invariants (transition invariants)
	Matrix representation

8.2 Reachability Graph (Bounded Nets Only)

- **What it represents**: every possible configuration of tokens reachable starting from M_0.
- **Nodes** $\rightarrow$ reachable markings $M \in R(M_0)$
- **Arcs** $\rightarrow M \xrightarrow{t} M'$ if and only if $M[t\rangle M'$
- It acts as the finite state machine (FSM) equivalent to the net structure.





Immediately Verifiable Properties

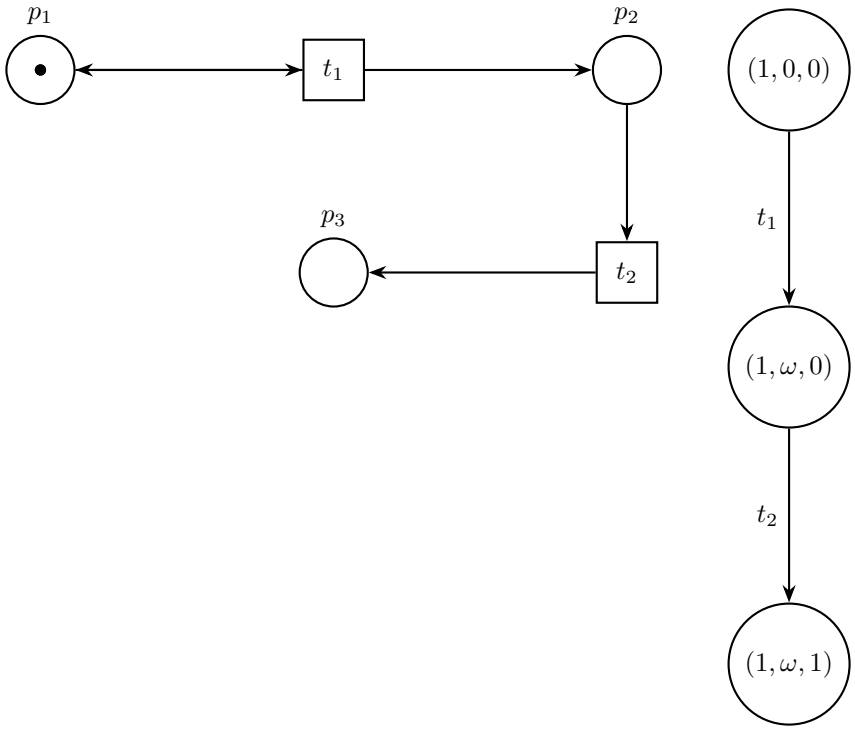
- **Deadlock** $\rightarrow$ node without outgoing arcs
- **Liveness** $\rightarrow$ every transition appears as a label
- **Reversibility** $\rightarrow$ strongly connected graph
- **Mutual Exclusion** $\rightarrow$ no nodes satisfying $M(p_i) + M(p_j) > 1$

8.3 Coverability Graph — Works Even on Unbounded Nets

When the net is unbounded, the reachability graph contains infinite nodes. The coverability graph compresses infinite markings using ω (≪controlled infinity≫).

M covers M' $\iff \forall p\, M(p) \geq M'(p)$

Key difference in the algorithm When generating a new marking M', if there exists M'' along the path from the root such that $M' \succeq M'' \rightarrow$ we set ω where $M'(p) > M''(p)$.



Why This Example is Correct (Step-by-Step)

Initial marking: $M_0 = (1, 0, 0)$

1. From $(1, 0, 0)$, only t_1 is enabled $\rightarrow$ t_1 fires:

- consumes 1 from p_1
- produces 1 in p_2
- produces 1 in p_1

$\rightarrow$ new marking $(1, \omega, 0)$ this is because:

- $M_1(1) = 1 \geq 1$ (M_0) (marking 1 place 1 = 1)
- $M_1(2) = 1 > 0$ (M_0) hence, since $1 \geq 1$ holds (for place 1), ω is introduced
- $M_1(3) = 0 \geq 0$ (M_0)

In words:

- Inspect Place 2: "Hey, here I went from 0 to 1. That's an increase! Could it be an infinity?"
- Check other places (e.g., Place 1): "To be a true infinity, I must not have 'paid' for this increase by draining other places. Let's look at Place 1... ah, it contained 1 and it still contains 1 ($1 \geq 1$). Excellent, Place 1 has not worsened."

Summarizing, two rules must be followed:

1. I verify the current marking by comparing it with ALL markings moving back towards the root.
2. For each place, if it has not worsened but remained at least equal, and at least 1 has improved, then ω is placed in the improved ones.

What Can Be Concluded with Certainty from This Analysis

- No ω appears $\implies$ the net is bounded (the coverability graph coincides exactly with the reachability graph)
- ω appears $\implies$ there exists at least one unbounded place (here p_1 is unbounded)
- **We cannot prove liveness** (the presence of ω hides information: a transition might appear infinitely enabled without truly being so; indeed, if a node with a transition generating infinite tokens also has a transition that only removes them, the postset of the first transition will no longer be ω, but the graph is unable to track this)
- Every marking without ω in the coverability graph is guaranteed to be reachable from the initial marking

Golden Rule

Reachability Graph $\rightarrow$ complete analysis (only if bounded)

Coverability Graph $\rightarrow$ partial analysis but always terminating

9 —

Matrix Representation—

Instead of drawing the net, we describe it using numbers: this is the most powerful method for automated analysis.

- $\mathbf{I} \in \mathbb{N}^{|P|\times|T|}$ **input matrix (pre-conditions), P: Places, T: Transitions**

$$I[i,j] = \text{weight of arc } p_i \to t_j \quad (0 \text{ if the arc does not exist})$$

- $\mathbf{O} \in \mathbb{N}^{|P|\times|T|}$ **output matrix (post-conditions)**

$$O[i,j] = \text{weight of arc } t_j \to p_i \quad (0 \text{ if the arc does not exist})$$

- $\mathbf{C} = O - I$ **incidence matrix** (the most critical!)

$$C[i,j] = \begin{cases} +w & \text{if firing } t_j \text{ adds } w \text{ tokens to } p_i \\ -w & \text{if firing } t_j \text{ consumes } w \text{ tokens from } p_i \\ 0 & \text{otherwise (no arc or bidirectional arc with equal weight)} \end{cases}$$

- $\mathbf{m} \in \mathbb{N}^{|P|}$ **column vector of the current marking**

$$m[i] = \text{number of tokens currently in place } p_i$$

- $\mathbf{s} \in \mathbb{N}^{|T|}$ —firing count vector— (firing vector)

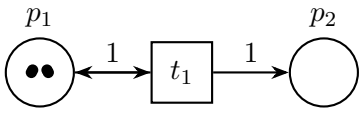
$$s[j] = \text{how many times transition } t_j \text{ fired within the sequence}$$

State Equation (Evolution Equation):

$$\boxed{m' = m + Cs}$$

Meaning: the new marking corresponds to the old one plus the net effect of the firing sequence s.

Concrete example:



I	t_1
p_1	-1
p_2	0

O	t_1
p_1	$+1$
p_2	$+1$

$$\implies \quad C = \begin{bmatrix} 0 \\ +1 \end{bmatrix}$$

9.1 P-invariants (Place Invariants)

A **P-invariant** is a row vector $h \in \mathbb{Z}^{|P|}$, non-null, satisfying:

$$\boxed{h^T C = 0}$$

This acts as a **necessary** and **sufficient** condition for h to be a **P-invariant**.

Physical Meaning

The scalar $h^T m$ represents a **weighted token quantity** and remains **invariant** during the entire execution of the net:

$$\boxed{\forall m' \text{ reachable from } m_0 \quad \Rightarrow \quad h^T m' = h^T m_0 = \text{constant}}$$

Real-World Examples

P-invariant	Net	Physical Meaning
$h = [1\ 1\ 0\ 0\ 0]$	Readers/Writers	ReadyReaders + ActiveReaders = constant
$h = [0\ 0\ 1\ 1]$	Writers	ReadyWriters + ActiveWriters = constant
$h = [1\ 0\ 1]$	Semaphore	"free" + "occupied" places = 1 always
$h = [1\ 1\ 1]$	Conservative system	Total number of tokens remains constant

Theorems

Condition on P-invariants	Consequence (proven)
There exists $h \geq 0$ (semi-positive)	All places with weight $h(i) > 0$ are **bounded**
There exists $h > 0$ (strictly positive weights)	The net is **conservative** $\Rightarrow$ **bounded**
An ensemble of semi-positive P-invariants exists such that every place has a weight > 0 in at least one of them	The net is **bounded** (P-invariant coverage theorem)

How are P-invariants found in practice?

Farkas Algorithm — Step-by-Step

Let C be the incidence matrix ($|P| \times |T|$).

1. Construct the initial matrix (by concatenating the identity matrix to the right of C):
$$D_0 = \begin{bmatrix} C & | & I_{|P|} \end{bmatrix}$$
2. For each column $j = 1, \ldots, |T|$ of part C:
   - Select all row pairs r_i, r_k of D_{j-1} such that: $D_{j-1}[i,j] \cdot D_{j-1}[k,j] < 0$ (rows that can be linearly combined)
   - For each pair:
   $$\text{append a new row at the bottom} = |D_{j-1}[i,j]| \cdot r_k + |D_{j-1}[k,j]| \cdot r_i$$
   (result of linear combination). Divided by the **GCD** of its elements (to keep numbers small).
3. Delete from the matrix all rows that **still** have a non-zero value in column j.
4. Proceed iteratively until matrix C is exhausted.
5. At the end, you obtain $D_{|T|}$. **The last $|P|$ columns of $D_{|T|}$ contain a base of semi-positive P-invariants**.

Classical Example

Net with incidence matrix:
$$C = \begin{bmatrix} -1 & 1 \\ 1 & -1 \\ -1 & 1 \end{bmatrix} \qquad \text{(3 places, 2 transitions)}$$

Applying Farkas:
$$D_0 = \begin{bmatrix} -1 & 1 & 1 & 0 & 0 \\ 1 & -1 & 0 & 1 & 0 \\ -1 & 1 & 0 & 0 & 1 \end{bmatrix}$$

Column 1: pairs (1,2) and (2,3) $\rightarrow$ new rows: (0,0,1,1,0) and (0,0,0,1,1). After deletion and normalization $\rightarrow$ next step...

At the end (when C is exhausted), the solutions obtained are:
$$h = [1\ 1\ 0]$$
$$h = [1\ 0\ 1]$$

Flash Summary of the Farkas Algorithm

Objective	Find **semi-positive** P-invariants
Input	Incidence matrix C
Key step	Linear combinations of rows with opposite signs
Final result	Base of P-invariants ≥ 0 in the last columns
Conclusion	If every place has a weight > 0 in at least one invariant $\rightarrow$ net is bounded

The Farkas algorithm is useful because it directly searches for semi-positive invariants, allowing immediate conclusions on net boundedness without constructing the coverability graph.

9.2 T-invariants (Transition Invariants)

A **T-invariant** is a column vector $x \in \mathbb{N}^{|T|}$, non-null, satisfying:

$$\boxed{Cx = 0}$$

Physical Meaning

There exists a firing sequence whose —net effect equals zero—:

$m + Cx = m \quad \Rightarrow \quad$ if the sequence is feasible, you return exactly to the initial marking

(indicates the firing count required for each single transition, but not their specific order)

Fundamental Theorem

Liveness Theorem (or T-invariant Coverage Theorem)

If a net is:

1. **bounded**
2. **covered by T-invariants** (i.e., there exists a linear combination with positive coefficients of T-invariants yielding a strictly positive vector)

$\Rightarrow$ the net is **live** (every transition can fire infinitely often from any reachable state).

How They Are Calculated

We solve $Cx = 0 \rightarrow$ we search for possible values that annihilate the transition effects of C.

9.3 Definitive Summary (The table to print and hang on the wall)

Characteristic	**P-invariant** h	**T-invariant** x
Equation	$h^T C = 0$	$Cx = 0$
Vector type	row	column
What remains constant	weighted token quantity	cyclic sequences
If a strictly positive exists	conservative net $\rightarrow$ bounded	conservative only if also bounded
If they cover the whole space	bounded net	live net (if also bounded)
Useful to prove	boundedness, mutual exclusion	liveness, protocol correctness

WARNING:

- the existence of a **strictly positive P-invariant** is a necessary and sufficient condition for the net to be conservative.
- The existence of a **T-invariant** is a necessary but **NOT** sufficient condition to prove **validity**, since we must guarantee that the firing sequence is feasible (the T-invariant states that if they all fire, the initial state is restored, but it doesn't guarantee whether token availability allows them to fire).

10 Forbidden State Control (Linear Constraint Controller)

Objective: **prevent the net from reaching undesirable markings** (forbidden states) by adding places and arcs only (without altering original transitions).

Core Idea

We want to enforce that certain linear token combinations **never exceed a threshold**:

$$\boxed{L\,m \leq b}$$

where:

- L = P-invariant constraint vector
- m = current marking
- b = maximum threshold number (total weighted token sum of the net)

10.1 Forbidden State Control (Linear Constraint Controller)

Objective: prevent the net from entering forbidden markings of the form:

$$\boxed{Lm \leq b}$$

As seen in Operations Research, any system of linear inequalities can be transformed into a system of equations by introducing non-negative **slack variables**:

$$Lm + x = b \qquad x \geq 0$$

Each component of x becomes a **control place** (monitor place, slack place).

Core Idea

The control place p_c initially contains $b - Lm_0$ tokens and, whenever the system approaches the constraint boundary, the control place loses tokens. Upon reaching 0, it **blocks** transitions that would violate the constraint.

The Two Formulas to Know

Let C_s be the incidence matrix of the original net structure.

$$\boxed{C_c = -LC_s} \qquad (C_c \text{ is matrix C augmented with the row of the new place } p_c) \tag{1}$$

$$\boxed{m_{0c} = b - Lm_{0s}} \qquad (\text{initial marking of the new control places}) \tag{2}$$

m_{0s} indicates the initial marking without the control place present.

These two formulas work universally, for any linear constraint $Lm \leq b$.

Example 1 — Simple Mutual Exclusion

We want to guarantee that two processes are never simultaneously within their critical section:

$$m(p_1) + m(p_3) \leq 1$$

Constraint: $L = \begin{bmatrix} 1 & 0 & 1 & 0 \end{bmatrix}$, $b = 1$
Suppose:

$$C_s = \begin{bmatrix} 0 & -1 & 1 & 0 \\ 1 & 0 & 0 & -1 \\ 0 & 1 & 0 & -1 \\ -1 & 0 & -1 & 1 \end{bmatrix}$$

Then:

$$C_c = -LC_s = \begin{bmatrix} 0 & 0 & -1 & 1 \end{bmatrix} \qquad m_{0c} = 1 - Lm_{0s} = 1$$

meaning:

- C_c indicates how the complementary place must behave to respect $M(p_s) + M(p_c) = b$ (note that it is L1*C11 + L2*C21 + L3*C31 + L4*C41, L1*C21 + L2*C22...)
- m_{0c} indicates the initial marking of place p_c (computed as constraint b - L*initial marking)

Summary Table

Task	Action
Enforce $Lm \leq b$	Add control places
Compute arcs	$C_c = -LC_s$
Compute initial tokens	$m_{0c} = b - Lm_{0s}$
Result	The constraint is always respected and behavior is maximally permissive

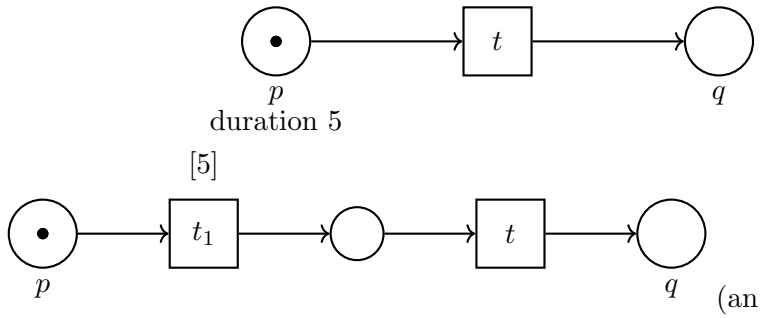
11 Timed Petri Nets – Time Basic (TB)

Motivation: **Hard Real-Time** systems in certain cases, if a temporal constraint is violated, catastrophes can occur; hence, a deterministic model with explicit time is required → **Time Basic Petri Nets**.

11.1 Temporal Models – Overview

Time assigned to	Places	Transitions
Meaning	minimum residence duration	duration or firing instant
Equivalence	**Yes** – interchangeable	

Equivalence Example (time on place → time on transition)



(an additional transition t1 is added)

Core Concepts – Detailed Explanation

1. **Tokens are distinguishable and carry a timestamp** Each created token carries a value $\tau_i \in \Theta$ (usually $\mathbb{R}_{\geq 0}$ or $\mathbb{N}$) representing its precise instant of creation. Two tokens with distinct timestamps are **distinct**, even if residing in the same place.

2. **Enabling Time (of a transition)**

$$\boxed{en(t) = \max\{\tau_1, \tau_2, \ldots, \tau_k\}}$$

   where $\tau_1, \ldots, \tau_k$ are the timestamps of the tokens chosen to form the enabling tuple of t. *Intuition*: the transition must wait until **all** necessary tokens have arrived.

3. **Time Function** tf_t Each transition t features a function:

$$tf_t : \Theta^k \longrightarrow 2^{\Theta}$$

   returning the set of possible firing times. In practice, a closed interval is almost universally employed:

$$tf_t(en) = [eft_t(en),\ lft_t(en)] \quad (lft_t \text{ can be } \infty)$$

   subject to the mandatory condition:

$$\boxed{eft_t(en) \geq en(t)}$$

4. **Firing Rule** An enabled transition t can fire at any instant:

$$\theta \in [eft_t(en),\ lft_t(en)]$$

   All tokens produced by that firing will carry **exactly** the timestamp θ.

5. **Initial Marking** m_0 No longer a vector of integers, but a multiset of pairs (τ, n) for each place:

$$m_0(p_1) = \{(0, 3),\ (5, 1)\} \quad (3 \text{ tokens created at time 0, 1 token at time 5})$$

Formal Definition

$$\textbf{Time Basic Net} = \boxed{\langle P,\ T,\ \Theta,\ F,\ tf,\ m_0 \rangle}$$

- P, T, F are identical to classical Petri nets.
- Θ corresponds to the time domain, i.e., the numerical set containing the representation of time instants.
- tf is a mapping associating each transition t $\in$ T with a time function tf_t which, given an input enabling tuple en (the set of timestamps of tokens selected for enabling within the preset), returns a set of possible firing times: where $tf : T \to (\Theta^k \to 2^{\Theta})$.
- m_0 is the initial marking, but where tokens store timestamp information.

Examples of Common Time Functions

Type	Notation	Physical Meaning
Fixed	$[3, 3]$	fires exactly after 3 units
Minimum Delay	$[2, \infty)$	can fire after at least 2 units (weak)
Window	$[4, 8]$	must fire between 4 and 8 units (e.g., enabled at 10, fires between 14 and 1
Relative	$[en + 1, en + 5]$	a window shifting alongside enabling time
Absolute	$[10, 15]$	fires exclusively between absolute time 10 and 15

Practical Example

Net: $p_1 \xrightarrow{t\ [2,5]} p_2$ with two tokens in p_1 created at time 0.

- $en(t) = \max(0, 0) = 0$
- t can fire at any $\theta \in [2, 5]$
- Suppose $\theta = 4 \rightarrow$ the token created in p_2 will carry timestamp 4
- If a transition t_2 exists consuming p_2 with $tf_{t_2} = [\theta + 1, \theta + 3]$, then t_2 can fire within $[5, 7]$

11.2 Evolution – The 4 Temporal Semantics

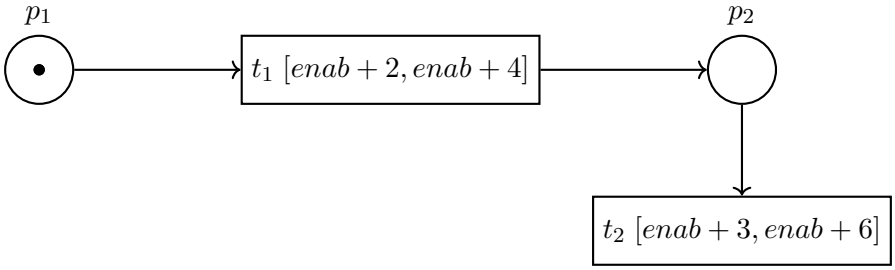
Semantics	A1	A2	A3	A4	A5	**Transition Behavior**
WTS (weak)	✓		✓			can **never** fire
MWTS (monotonic weak)	✓	✓	✓			can **never** fire
STS (strong)	✓	✓	✓	✓	✓	**must** fire within lft
Mixed	transitions marked "W" = weak					transitions without "W" = strong

- **A1 – Monotonicity with respect to initial marking** All future firing times must be $\geq$ than the maximum timestamp within the initial marking (of tokens consumed by the transition). *Tokens cannot be created in the past.* **Example**: if at startup there is a token with $\tau = 10$, no transition requiring that token can ever fire before time 10.
- **A2 – Monotonicity of the firing sequence** Firing times within the sequence must be non-decreasing: $\theta_1 \leq \theta_2 \leq \theta_3 \leq \ldots$ *The global time of the net can never roll back.* **Example**: sequence t_1 at 12 $\rightarrow$ t_2 at 8 $\rightarrow$ t_3 at 15 is invalid.
- **A3 – Non-Zeno (time divergence)** It is impossible to have infinite firings within a finite time interval. *No infinite instantaneous loops.* **Example**: a transition with $[0, 0]$ (min time, max firing time) in a loop is invalid.
- **A4 – Strong initial marking** In the initial marking, for each transition already enabled, its lft (Latest Firing Time) must be $\geq$ than the maximum initial timestamp. *A transition cannot exist with an lft strictly lower than the timestamp of any token enabling it (unlike **A1**, this guarantees validity of the initial state, whereas A1 ensures firing happens later).* **Example**: token with $\tau = 15$ and enabled transition with $lft = 12 \rightarrow$ invalid initial marking in STS.
- **A5 – Strong firing sequence** If a transition fires at time θ, no other enabled transition must exist satisfying $lft < \theta$ (unless that unfired transition was disabled beforehand). *In practice: if a transition has a finite lft and can fire, it **must** do so eventually.* **Example**: $p_1 \xrightarrow{t_1\ [2,\infty)} p_2 \xrightarrow{t_2\ [3,enab+6)} p_3$ Token in p_1 at time 0 $\rightarrow$ t_1 fires at 4 $\rightarrow$ t_2 enabled at 4 with $lft = 10$. In STS, t_2 **must** fire by time 10!

Core Theorems:

- Any feasible sequence in WTS can be reordered into an equivalent feasible sequence in MWTS $\Rightarrow$ we can **ignore A2** during analysis (we universally rely on local weak semantics).
- By virtue of the last axiom, one might think a transformation exists to map any MWTS firing sequence into an STS sequence, establishing equivalence. Unfortunately, this is not true: an STS sequence is always an MWTS sequence, but the converse does not always hold.

Practical Example: WTS vs STS



Initial timestamp: $\tau(p_1) = 0$

- t_1 enabled at 0 (enabled but can only fire at 2) $\rightarrow$ fires (e.g.) at 3 $\rightarrow$ new token in p_2 with $\tau = 3$
- t_2 enabled at 3 $\rightarrow$ can fire within $[6, 9]$
- **WTS/MWTS**: t_2 can choose never to fire (since A5 does not apply)
- **STS**: t_2 **must** fire by time 9

11.3 —

Mixed Time Semantics—

When the system contains both deterministic and non-deterministic components (human choices on whether to fire or not), enforcing a uniform semantics on the whole net is restrictive.

Definition

Semantics are assigned to **each individual transition**:

- —Strong transitions— (default): must fire by their lft if they remain enabled
- —Weak transitions—: can fire at any point within their interval (or choose never to do so)

Graphical Convention: weak transitions are denoted with a **W**, strong by default otherwise.

Classical Example

Net:

$$p_1 \xrightarrow{t_1\ [1,\infty)\ \mathrm{W}} p_2 \xrightarrow{t_2\ [enab+3,10]} p_3$$

- a token in p_1 created at time 0
- t_1 is weak (labeled W)
- t_2 is strong (no W $\rightarrow$ strong by default), $lft = 10$

how long can t_1 wait?:
t_1 must fire **at the latest by time 7**, because:

- if t_1 fires at time θ, then p_2 receives the token at time θ
- t_2 becomes enabled at time θ
- t_2 is strong and must fire by absolute time 10
- its $eft = 3$, meaning $\theta + 3$
- but it must satisfy $\leq 10 \rightarrow \theta + 3 \leq 10 \rightarrow \theta \leq 7$

If t_1 fired after time 7 (e.g., at 8), t_2 would be enabled at 8 and its eft would become $8 + 3 = 11$, but the transition has an absolute lft of 10 $\rightarrow$ it could no longer meet the deadline $\rightarrow$ violation of A5.

12 Are Time Basic Nets Reducible to High-Level Nets?

is it possible to translate a Time Basic net into a high-level net (Colored/High-Level Petri Net), thereby reusing existing analysis techniques?

Attempted Translation: TB → High-Level Nets

Idea:

- Each token carries a variable `chronos` = creation timestamp
- Predicates check timestamps and $[eft, lft]$ intervals
- Actions assign the firing time θ to the new tokens

This works perfectly for the syntactic part.

The Real Problem: Strong Semantics

Axiom	Modelable in High-Level Nets?	Notes
A1, A3 (base)	Yes	local timestamp checks
A2 (monotonicity)	Yes	global place "current time"
A4 and A5 (strong)	NO	require global knowledge of the marking

A5 is the killer: to know if a transition can fire right now, it must know whether **another** transition in the net is about to miss its lft. ⇒ it would require every predicate to see the **entire** net marking → violates locality → impractical.

Conclusion ≪Time Basic nets **are not reducible** to high-level nets because strong semantics requires global state knowledge, destroying the locality characteristic of Petri nets. For this reason, specific analysis algorithms have been developed (symbolic states + Floyd + inclusion + etc.).≫

12.1 High-Level Time Petri Nets (HLTPN)

Since reduction is impossible, the most powerful model is born: **High-Level Time Petri Nets (HLTPN)** = high-level nets + Time Basic explicit time.

Main characteristics:

- **High-Level:** Firing intervals can depend on token content
- **timed:** extension creating predicates such that a transition fires not just based on token presence but whether they satisfy the constraints.
- **petri nets:** core foundational structure upon which extensions are added

Critical Exam Theorem The dominant complexity in HLTPN analysis **always** stems from the temporal domain. Anyone capable of analyzing Time Basic nets can analyze HLTPNs (the functional part reduces via classical techniques: unfolding, coverability graphs, etc.).

13 Temporal Reachability Analysis in Time Basic Nets

In classical nets, the reachability graph is finite (or utilizes ω). In Time Basic nets, it is **always infinite** for two fundamental reasons:

1. Each transition can fire at infinite distinct time instances → infinite timestamps
2. Even with no enabled transitions, time keeps advancing

⇒ a completely new approach is required.

13.1 Symbolic States $[\mu, C]$ – The Solution

Instead of numerical timestamps, we use **symbols** (τ_i) + **constraints** (C).

A symbolic state is a pair $[\mu, C]$ where:

- $\mu(p)$ = multiset of symbols τ_i in each place
- C = set of inequalities among τ_i (e.g., $\tau_0 < \tau_1 \leq \tau_0 + 5$)

Classical Example

Net with three places: - token in p_1 and p_3 created at time 0 - token in p_2 created at time 1

Symbolic state:

$$\mu(p_1) = \tau_0, \quad \mu(p_2) = \tau_1, \quad \mu(p_3) = \tau_0$$

$$C = \{\tau_0 < \tau_1\}$$

13.2 Formula to Generate the New Symbolic State

When transition t fires within a state $[\mu, C]$:

1. Introduce a new symbol τ_{new}
2. Update the marking μ' (remove preset, add postset with τ_{new})
3. Compute the new constraints C':

Complete Practical Example

Net: $p_1 \xrightarrow{t\ [2,5]} p_2$, token in p_1 created at time 0

Initial state: μ_0: $p_1 = \tau_0$, C_0: $\tau_0 = 0$

After firing t: μ_1: $p_2 = \tau_1$ C_1: $\tau_0 = 0 \ \wedge \ \tau_1 \geq \tau_0 \ \wedge \ 2 \leq \tau_1 \leq 5 \Rightarrow C_1$: $2 \leq \tau_1 \leq 5$

WARNING:

repeating the steps to find new states reveals that the algorithm never terminates; indeed, determining whether a state has been explored is difficult, and coverage of all states is not guaranteed. Nonetheless, this method is useful to inspect net evolution and check for consistency within a given time frame.

13.3 From Infinite Trees to Finite Graphs – The 4 Contraction Techniques

The symbolic reachability tree is **always infinite** (even for trivial nets). Time does not roll back → **no cyclic graph can exist**, but we want at least a more compact **finite acyclic graph**.

To achieve this, we must **forget the history** of how we reached a state and recognize that two distinct symbolic states can, in reality, **represent the identical future evolution**.

The four techniques (to be known in this specific order) are:

1. Floyd Algorithm – Eliminating "Ghost" Constraints

Each new state inherits all constraints of its predecessor → two states with the same marking but different paths feature distinct constraints → appearing different.

Many constraints involve symbols τ_i **already consumed** (no longer present). We cannot simply delete them: they might express **indirect** constraints on variables that are still live.

Classical Example:

Constraints: $B - A \leq 5$ $C - B \leq 6$ where B is dead (no longer in the marking)

Summing yields: $C - A \leq 11$ → a constraint between A and C that must be preserved!

Floyd-Warshall Algorithm

1. **Constraint Normalization:**

- Rewrite all temporal constraints (derived from transition durations or deadlines) into the **canonical form**:
$$T_X - T_Y \leq k$$
- This inequality defines a directed edge in the constraint graph: an edge from **Y** to **X** with weight k.
- **Example:** The constraint $T_X \geq T_Y + \tau$ (i.e., $T_X - T_Y \geq \tau$) converts to $T_Y - T_X \leq -\tau$, creating the edge $X \xrightarrow{-\tau} Y$.

2. **Adjacency Matrix Construction:**
   - Initialize the **Constraint Matrix** M, where rows and columns correspond to all **temporal variables** (live and dead firing instants).
   - Set $M[i,j] = k$ for each direct constraint $T_i - T_j \leq k$.
   - Set $M[i,j] = \infty$ if no direct constraint exists between j and i.
   - Set $M[i,i] = 0$.
3. **Execution of the Floyd–Warshall Algorithm:**
   - Apply the Bellman equation, iterating over all possible intermediate nodes **k**:
$$M[i,j] \leftarrow \min\big(M[i,j],\ M[i,k] + M[k,j]\big)$$
   - **Role of $M[i,j]$:** At the end, $M[i,j]$ will contain the **shortest path** (the most stringent temporal constraint) from node **j** to node **i**. This enforces the inequality $\mathbf{T_i - T_j \leq M[i,j]}$.
   - **Negative Cycles Check:** If $M[i,i] < 0$ for any i, the constraint system is **contradictory** and admits no feasible solutions.
4. **Final Matrix Reduction:**
   - Extract from the resulting matrix M **only the rows and columns** corresponding to the temporal variables of transitions that are **still live** and relevant for future analysis.

After Floyd, two states with the same μ and identical constraints (even if reached via different paths) become **equal**.

2. State Class Inclusion

Even after Floyd, we can have identical states except for a constant factor.

Example: net with a loop adding +1 each cycle → constraints: $\tau_1 \leq 5,\ \tau_1 \leq 6,\ \tau_1 \leq 7,\ \ldots$

All have the same μ, but increasingly wider constraints.

Inclusion Definition A symbolic state $S_1 = [\mu_1, C_1]$ is **contained** within $S_2 = [\mu_2, C_2]$ ($S_1 \sqsubseteq S_2$) if and only if:

- $\mu_1 = \mu_2$
- $C_1 \implies C_2$ (the constraints of S_1 are more restrictive)

Graph rule: **keep only the more general state** (the one with wider constraints).

3. Relative Times: Temporal Abstraction

The Relative Times (or Differential Abstraction) principle is a reduction technique for the state space of Timed Petri Nets (TPNs) when net evolution is independent of a global clock.

—

Problem: State Explosion with Absolute Times

In TPNs, every state (marking M) is defined by a set of tokens and their **absolute timestamps** (ts). Maintaining a reference to an absolute clock t_{absolute} causes state-space explosion.

- Two states differing only by a constant offset in time are treated as distinct, even if their dynamic evolution will be identical.
- **Example:** A token created at time $t = 0$ or at time $t' = 1000$ leads to the **identical future evolution** if transition constraints are purely relative.

—

Principle: Elimination of Absolute References

Objective: Future evolution depends exclusively on the **differences** (or delays) between timestamps, not on their absolute value.

—

Transformation Technique

To eliminate dependence on absolute time, a —translation of the temporal zero— is operated at each transition firing:

1. **Reference Choice (τ_{ref}):** A **reference time** τ_{ref} is identified in the current marking, often the **minimum** or **maximum** of current timestamps, or the time of the next inevitable event.
2. **Relative Constraints Recalculation:** All timestamps ts_i and temporal constraints are **rewritten** in relation to τ_{ref}:
$$ts'_i = ts_i - \tau_{\text{ref}}$$
3. **Elimination of Reference to τ_{ref}:** Since future evolution depends on differences, the reference instant τ_{ref} (which becomes zero in the relative system) can be ignored.
4. **Result:** States that previously differed only by a temporal offset (Δt) are **identified as equivalent**. This drastically reduces the number of distinct states in the reachability graph, making it checkable.

Practical Example: The Self-Cyclic Queue

Objective: Demonstrate that the net can evolve infinitely without generating infinite states in the reachability graph, by ignoring absolute time.

—Net (Implied Time Basic PN):—

$$P_1 \quad \xrightarrow{T_1\ [\tau_1,\tau_2]} \quad P_1 \quad \text{Initial Marking: } M_0(P_1) = 1$$

- Place P_1 contains one token.
- Transition T_1 consumes and produces a token from P_1.
- T_1 constraint: Fires in a relative interval **after** enabling. For instance, $[1, 2]$. (Must fire between 1 and 2 time units after it is enabled).

Scenario 1: Analysis with Absolute Times (The problematic approach)

1. M_0**:** P_1 has a token g_0 with $ts(g_0) = 0$.
2. T_1 **fires:** T_1 is enabled at $t = 0$. It must fire within $[0 + 1, 0 + 2] = [1, 2]$. Suppose it fires at $t_1 = 1.5$.
3. M_1**:** P_1 has a new token g_1 with $ts(g_1) = 1.5$.
4. T_1 **fires again:** T_1 is enabled at $t = 1.5$. It must fire within $[1.5+1, 1.5+2] = [2.5, 3.5]$. Suppose it fires at $t_2 = 2.8$.
5. M_2**:** P_1 has a new token g_2 with $ts(g_2) = 2.8$.

Since $ts(g_1) \neq ts(g_2)$, states M_1 and M_2 are considered **distinct** in the absolute analysis. Any choice of firing time creates a new state $\rightarrow$ **Infinite states**.

—

Scenario 2: Analysis with Relative Times (The correct approach)

The **eliminate absolute references** technique is applied by choosing the token timestamp as τ_{ref} (the new temporal zero).

1. M_0**:** P_1 contains g_0 with $ts(g_0) = 0$. The reference time is $\tau_{\text{ref}} = 0$. The token has $ts' = 0$.
2. T_1 **fires at** t_1 **(Relative):** T_1 fires within $[1, 2]$ (delay Δt).
3. M_1 **(Translation):** The new token g_1 is produced with $ts(g_1) = t_1$. **Time is translated:** t_1 is selected as the new τ_{ref}.
$$ts'(g_1) = ts(g_1) - t_1 = 0$$
4. **Future Evolution:** State M_1 is identified as a state where P_1 has a token with $ts' = 0$, which is **identical** to the initial state M_0.

Result: The net features **only a single state within the relative reachability graph** ($M_0 \equiv M_1 \equiv M_2 \dots$), proving that evolution can be infinite without state explosion.

4. Anonymous Tokens τ_A

If the timestamp of a token **no longer influences any future decision** (unnecessary to compute any eft or lft), we can **erase it**.

13.4 Summary of the 4 Techniques (Table to Learn)

Technique	What it eliminates	Effect on the graph
Floyd	indirect constraints on dead symbols	states with distinct histories coincide
Inclusion	states with more restrictive constraints	eliminates states that are “too narrow”
Relative times	references to 0, 10, 100...	time-shifted states coincide
Anonymous tokens τ_A	timestamps irrelevant to the future	infinite loops become finite